```
$ http POST :9002/orders isbn=1234567891 quantity=3

HTTP/1.1 200 OK
{
  "bookIsbn": "1234567891",
  "bookName": "Northern Lights - Lyra Silverstar",
  "bookPrice": 9.9,
  "createdDate": "2022-06-06T09:59:32.961420Z",
  "id": 2,
  "lastModifiedDate": "2022-06-06T09:59:32.961420Z",
  "quantity": 3,
  "status": "ACCEPTED",
  "version": 1
}
```

When you are done verifying the interaction, stop the applications with Ctrl-C and the container with `docker-compose down`.

That concludes our implementation of the order-creation logic. If the book exists in the catalog, the order will be accepted. If an empty result is returned, it's rejected. But what if Catalog Service takes too much time to reply? What if it's momentarily unavailable and can't process any new requests? What if it replies with an error? The following section will answer and handle all these questions.

8.4 Resilient applications with Reactive Spring

Resilience is about keeping a system available and delivering its services, even when failures happen. Since failures will happen, and there's no way to prevent them all, it is critical to design fault-tolerant applications. The goal is to keep the system available without the user noticing any failures. In the worst-case scenario, the system may have degraded functionality (*graceful degradation*), but it should still be available.